

```

#!/usr/bin/env python3
"""
Skill Packager - Creates a distributable .skill file of a skill folder

Usage:
    python utils/package_skill.py <path/to/skill-folder> [output-directory]

Example:
    python utils/package_skill.py skills/public/my-skill
    python utils/package_skill.py skills/public/my-skill ./dist
"""

import fnmatch
import sys
import zipfile
from pathlib import Path
from scripts.quick_validate import validate_skill

# Patterns to exclude when packaging skills.
EXCLUDE_DIRS = {"__pycache__", "node_modules"}
EXCLUDE_GLOBS = {"*.pyc"}
EXCLUDE_FILES = {".DS_Store"}
# Directories excluded only at the skill root (not when nested deeper).
ROOT_EXCLUDE_DIRS = {"evals"}

def should_exclude(rel_path: Path) -> bool:
    """Check if a path should be excluded from packaging."""
    parts = rel_path.parts
    if any(part in EXCLUDE_DIRS for part in parts):
        return True
    # rel_path is relative to skill_path.parent, so parts[0] is the skill
    # folder name and parts[1] (if present) is the first subdir.
    if len(parts) > 1 and parts[1] in ROOT_EXCLUDE_DIRS:
        return True
    name = rel_path.name
    if name in EXCLUDE_FILES:
        return True
    return any(fnmatch.fnmatch(name, pat) for pat in EXCLUDE_GLOBS)

def package_skill(skill_path, output_dir=None):
    """
    Package a skill folder into a .skill file.

    Args:
        skill_path: Path to the skill folder
        output_dir: Optional output directory for the .skill file (defaults to current
        directory)

    Returns:
        Path to the created .skill file, or None if error
    """
    skill_path = Path(skill_path).resolve()

```

```

# Validate skill folder exists
if not skill_path.exists():
    print(f"❌ Error: Skill folder not found: {skill_path}")
    return None

if not skill_path.is_dir():
    print(f"❌ Error: Path is not a directory: {skill_path}")
    return None

# Validate SKILL.md exists
skill_md = skill_path / "SKILL.md"
if not skill_md.exists():
    print(f"❌ Error: SKILL.md not found in {skill_path}")
    return None

# Run validation before packaging
print("🔍 Validating skill...")
valid, message = validate_skill(skill_path)
if not valid:
    print(f"❌ Validation failed: {message}")
    print("    Please fix the validation errors before packaging.")
    return None
print(f"✅ {message}\n")

# Determine output location
skill_name = skill_path.name
if output_dir:
    output_path = Path(output_dir).resolve()
    output_path.mkdir(parents=True, exist_ok=True)
else:
    output_path = Path.cwd()

skill_filename = output_path / f"{skill_name}.skill"

# Create the .skill file (zip format)
try:
    with zipfile.ZipFile(skill_filename, 'w', zipfile.ZIP_DEFLATED) as zipf:
        # Walk through the skill directory, excluding build artifacts
        for file_path in skill_path.rglob('*'):
            if not file_path.is_file():
                continue
            arcname = file_path.relative_to(skill_path.parent)
            if should_exclude(arcname):
                print(f"    Skipped: {arcname}")
                continue
            zipf.write(file_path, arcname)
            print(f"    Added: {arcname}")

    print(f"\n✅ Successfully packaged skill to: {skill_filename}")
    return skill_filename

except Exception as e:
    print(f"❌ Error creating .skill file: {e}")
    return None

```

```

def main():
    if len(sys.argv) < 2:
        print("Usage: python utils/package_skill.py <path/to/skill-folder>
[output-directory]")
        print("\nExample:")
        print("  python utils/package_skill.py skills/public/my-skill")
        print("  python utils/package_skill.py skills/public/my-skill ./dist")
        sys.exit(1)

    skill_path = sys.argv[1]
    output_dir = sys.argv[2] if len(sys.argv) > 2 else None

    print(f"📦 Packaging skill: {skill_path}")
    if output_dir:
        print(f"  Output directory: {output_dir}")
    print()

    result = package_skill(skill_path, output_dir)

    if result:
        sys.exit(0)
    else:
        sys.exit(1)

if __name__ == "__main__":
    main()

```